

WEATHER APP

Project Structure & Architecture Documentation

React + TypeScript + Vite | Task 4 (Lesson 4)

Reference sources: [Task Brief PDF](#) · [Links Vault Structure Guide](#) · [ui-challenge-one Project Structure Template](#)

1. Project Overview

1.1 Objective

Applications rarely operate in isolation — most exchange data with other systems through APIs. This project focuses on consuming weather data from a third-party API (WeatherAPI.com) and presenting it in a purpose-built React interface.

1.2 Scenario

Build a weather application that lets a user view the current weather for their detected location, and search for and save the weather of any other location. Locations must persist across sessions (via `localStorage`) so users are not forced to re-search each visit.

1.3 Goals of This Document

This document defines the folder structure, naming conventions, component architecture, state/data strategy, and tooling configuration to be followed while building the app. It synthesizes two prior reference architectures — a local-storage CRUD app ("Links Vault") and a section-composed marketing page ("ui-challenge-one") — into a single structure tailored to a weather application's needs (live data fetching, geolocation, persisted multi-location state, theming, and offline caching).

2. Requirements Summary

Condensed from the task brief. Full detail is in the attached PDF; this table is the working checklist for development and QA.

2.1 User Interface

- Intuitive, easy-to-use interface with a dedicated weather display page
- Show temperature, humidity, wind speed and related conditions clearly

2.2 Core Features

- Real-time current conditions, plus hourly and daily forecasts
- Toggle between hourly and daily forecast views
- Auto-detect user location (with permission) and allow manual search
- Show weather for both current and searched locations
- Push/in-app notifications for severe weather alerts
- Save and switch between multiple saved locations
- Theme customization (e.g. light / dark)
- Unit customization (Celsius vs Fahrenheit)
- Cached data available offline
- Fast load and smooth performance

- User data handled securely and privately

2.3 Evaluation Criteria (What Will Be Graded)

Category	Checkpoints
Design & UX	Intuitive layout, clear flow between functions, background/process notifications, consistent colors & typography
Functionality	Location detection, search, temperature/humidity/wind display, theme & unit switching, hourly/daily views
Interactivity	Pointer cursor + hover states on clickable elements, easy navigation
React/TS Usage	Custom, reused components; proper state; props sent/handled correctly; prop-driven variants over duplication
Code Quality	camelCase naming, self-explanatory variables, small functions, comments where useful
Responsiveness	Verified at 320px, 480px, 768px, 1024px, 1200px

3. Technology Stack

The stack follows the same base as both reference projects (React + TypeScript + Vite), with styling and library choices adapted to this task's requirements — notably that theming/customization is a graded requirement here, unlike the Links Vault project which forbade third-party CSS.

Layer	Choice
Framework	React 18/19, function components + hooks only (no class components)
Language	TypeScript, strict-ish (noUnusedLocals, noUnusedParameters on)
Build tool	Vite with @vitejs/plugin-react
Routing	react-router-dom v6 (per task instructions, for consistency across submissions)
Styling	CSS Modules (*.module.css) per component + one global src/index.css for design tokens/resets; Tailwind optional per task instructions but not required
Icons	react-icons (weather/UI glyphs)
Data source	WeatherAPI.com REST endpoints (current, forecast, search/autocomplete, alerts)
Persistence	Browser localStorage — saved locations, unit preference, theme preference, last-fetched cache for offline access
Geolocation	Browser Geolocation API, wrapped in a custom hook with permission/error handling
Notifications	Web Notifications API (severe weather alerts) + in-app toast component (background process feedback)
Linting	ESLint flat config: js recommended + typescript-eslint + react-hooks + react-refresh

Package manager	npm, package-lock.json committed
Module system	ESM only ("type": "module")

4. Folder Structure

This layout keeps Links Vault's separation of concerns (hooks / utils / types, one folder per component) and adds a pages/ layer and services/ layer, since this app is multi-view (dashboard + settings) and talks to a real external API rather than only localStorage.

```

weather-app/
├── index.html
├── package.json
├── package-lock.json
├── vite.config.ts
├── eslint.config.js
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
├── .gitignore
├── README.md
├── public/
│   ├── favicon.svg
│   └── icons.svg                # shared inline SVG sprite (weather glyphs)
├── src/
│   ├── main.tsx                # ReactDOM root render
│   ├── App.tsx                 # top-level layout + router outlet
│   ├── App.css                 # app-shell level styles
│   ├── index.css               # global reset + design tokens (:root vars, incl. themes)
│   ├── assets/
│   │   ├── icons/              # weather condition icon set
│   │   └── images/             # misc images (logo, backgrounds)
│   ├── types/
│   │   ├── weather.ts          # CurrentWeather, ForecastDay, HourlyForecast, Alert
│   │   ├── location.ts         # SavedLocation, SearchResult
│   │   └── settings.ts         # ThemeMode, UnitSystem
│   ├── utils/
│   │   ├── formatWeather.ts    # unit conversion, rounding, date/time formatting
│   │   ├── groupForecast.ts    # group hourly data into daily buckets
│   │   └── weatherAlerts.ts    # severity classification for alerts
│   ├── hooks/
│   │   ├── useLocalStorage.ts  # generic get/set/remove wrapper around localStorage
│   │   └── useGeolocation.ts   # requests & tracks browser geolocation, handles permission
│   ├── errors
│   │   └── useWeather.ts       # fetches current/forecast/alerts for a given location,
│   ├── caches result
│   │   ├── useSavedLocations.ts # add/remove/switch saved locations (persisted)
│   │   ├── useUnits.ts         # Celsius/Fahrenheit preference (persisted)
│   │   └── useTheme.ts         # light/dark theme preference (persisted)
│   ├── services/
│   │   └── weatherApi.ts       # thin fetch wrapper for WeatherAPI.com endpoints
│   ├── pages/
│   │   ├── Home/
│   │   │   ├── Home.tsx        # composes the weather dashboard for the active location
│   │   │   └── Home.module.css
│   │   └── Settings/
│   │       ├── Settings.tsx    # theme + unit preference screen
│   │       └── Settings.module.css
│   └── components/

```

```

├── Navbar/
│   ├── Navbar.tsx
│   └── Navbar.module.css
├── SearchBar/
│   ├── SearchBar.tsx
│   ├── SearchResultsList.tsx  # owned sub-component
│   └── SearchBar.module.css
├── CurrentConditions/
│   ├── CurrentConditions.tsx  # section: temp, humidity, wind, condition icon
│   └── CurrentConditions.module.css
├── ForecastView/
│   ├── ForecastView.tsx      # section: owns hourly/daily toggle state
│   ├── ForecastToggle.tsx    # sub-component: hourly/daily switch
│   ├── ForecastCard.tsx      # sub-component + exported type
│   └── ForecastView.module.css
├── LocationTabs/
│   ├── LocationTabs.tsx      # switch between saved locations
│   └── LocationTabs.module.css
├── AlertBanner/
│   ├── AlertBanner.tsx      # severe weather alert banner
│   └── AlertBanner.module.css
├── Toast/
│   ├── Toast.tsx             # background-process / status notifications
│   └── Toast.module.css
├── Inputs/
│   ├── Button.tsx            # generic Button primitive (variant prop)
│   ├── ToggleSwitch.tsx     # generic switch primitive (theme/units)
│   └── Input.module.css
├── Text/
│   ├── Text.tsx              # generic typography primitives (SectionHeading,
... )
│   └── Text.module.css

```

4.1 What Changed from the Reference Projects, and Why

Aspect	Reference Project	This Project
Data source	Links Vault: localStorage only	Adds services/ for the WeatherAPI.com client; localStorage now used for caching + preferences, not the source of truth
Pages	ui-challenge-one: single-page, section stack in App.tsx	Adds pages/ (Home, Settings) since the app needs distinct views, still composed the same section-based way
Styling	Links Vault: no third-party CSS allowed	Same rule kept — CSS Modules + tokens only — since the task allows but does not require a CSS library
Hooks	Both: one hook per cross-cutting concern	Extended with useGeolocation, useWeather, useSavedLocations, useUnits, useTheme

5. Naming Conventions

- Component folders and files: PascalCase (e.g. ForecastView/ForecastView.tsx)
- CSS Module files: <ComponentName>.module.css, imported as import styles from `"./ComponentName.module.css"`
- Hooks: camelCase, prefixed with use, inside src/hooks/ (e.g. useSavedLocations.ts)
- Variables and functions: camelCase, self-explanatory names (e.g. currentTemperatureCelsius, not temp1)
- Types/interfaces: PascalCase, colocated in src/types/ and imported where needed (e.g. SavedLocation, ForecastDay)
- One default export per component file; tightly-coupled sub-types use named exports from the same file (e.g. export type ForecastCardData from ForecastCard.tsx)
- Asset files: kebab-case, grouped into semantic subfolders under src/assets/

6. Component Architecture & Patterns

Every component follows the same shape used in the ui-challenge-one reference: a typed props interface, a single default-exported function component, and a co-located CSS module referenced as styles.xxx. Sections own their sub-components; generic primitives (Button, ToggleSwitch, Text) are imported, never duplicated.

6.1 Generic Primitive Example — Button

```
import type { ButtonHTMLAttributes, ReactNode } from "react";
import styles from "./Input.module.css";

interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: "primary" | "outline" | "ghost";
  children: ReactNode;
}

function Button({ variant = "primary", children, className, ...rest }: ButtonProps) {
  const variantClass =
    variant === "primary" ? styles.primary : variant === "outline" ? styles.outline :
    styles.ghost;

  return (
    <button className={` ${styles.button} ${variantClass} ${className ?? ""}` } {...rest}>
      {children}
    </button>
  );
}

export default Button;
```

6.2 Section Component Example — ForecastView

This mirrors Hero from the reference marketing-page project: a section owns local UI state (the hourly/daily toggle) and composes its own sub-components.

```
import { useState } from "react";
import ForecastToggle from "../ForecastToggle";
import ForecastCard from "../ForecastCard";
import type { ForecastDay, HourlyForecast } from "../../types/weather";
```

```
import styles from "../ForecastView.module.css";

interface ForecastViewProps {
  daily: ForecastDay[];
  hourly: HourlyForecast[];
}

function ForecastView({ daily, hourly }: ForecastViewProps) {
  const [mode, setMode] = useState<"daily" | "hourly">("daily");

  return (
    <section className={styles.forecast} aria-label="Forecast">
      <ForecastToggle mode={mode} onChange={setMode} />
      <div className={styles.cardRow}>
        {(mode === "daily" ? daily : hourly).map((entry) => (
          <ForecastCard key={entry.id} entry={entry} />
        ))}
      </div>
    </section>
  );
}

export default ForecastView;
```

6.3 App Shell

App.tsx stays a thin shell, same principle as the reference project — it composes the router and top-level layout (Navbar + page outlet), not page-specific logic.

7. State Management & Data Persistence

Stateful, reusable logic is extracted into hooks (src/hooks/), following the same rule as both reference projects: hooks accept sensible defaults and clean up all effects, and are never duplicated across components.

7.1 Data Flow

- useGeolocation resolves the browser's coordinates (or a permission-denied state)
- useWeather(locationQuery) fetches current + forecast + alert data for a location, and falls back to the localStorage cache when the network request fails (offline access requirement)
- useSavedLocations persists the list of saved locations and which one is active
- useUnits and useTheme persist the user's unit system and theme choice
- All persistence hooks are built on top of one generic useLocalStorage<T> wrapper, so storage read/write/error-handling logic exists in exactly one place

7.2 Example — useWeather (fetch + offline cache fallback)

```
import { useEffect, useState } from "react";
import { fetchCurrentAndForecast } from "../services/weatherApi";
import { useLocalStorage } from "../useLocalStorage";
import type { WeatherSnapshot } from "../types/weather";
```

```

export function useWeather(locationQuery: string) {
  const [data, setData] = useState<WeatherSnapshot | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [cache, setCache] = useLocalStorage<Record<string, WeatherSnapshot>>("weather-
cache", {});

  useEffect(() => {
    let isActive = true;

    async function load() {
      setIsLoading(true);
      try {
        const snapshot = await fetchCurrentAndForecast(locationQuery);
        if (!isActive) return;
        setData(snapshot);
        setCache({ ...cache, [locationQuery]: snapshot });
      } catch (err) {
        if (!isActive) return;
        setError("Could not fetch live weather – showing cached data if available.");
        setData(cache[locationQuery] ?? null);
      } finally {
        if (isActive) setIsLoading(false);
      }
    }

    load();
    return () => { isActive = false; };
  }, [locationQuery]);

  return { data, isLoading, error };
}

```

7.3 Local Storage Keys

Key	Shape	Purpose
weather-cache	Record<locationQuery, WeatherSnapshot>	Last known good data per location, for offline access
saved-locations	SavedLocation[]	User's saved/multiple locations
active-location-id	string	Which saved location is currently shown
unit-system	"metric" "imperial"	Celsius vs Fahrenheit preference
theme-mode	"light" "dark"	Theme preference

8. Design Tokens & Theming

Following the ui-challenge-one convention, all colors, fonts, radii and layout widths are defined once as CSS custom properties on `:root` in `src/index.css` and consumed via `var(--token-name)` — never hard-coded in component CSS modules. Theme switching is implemented by toggling a `data-theme` attribute on the document root (driven by `useTheme`), which swaps the token values shown below.


```

:root {
  --color-bg: #f5f7fa;
  --color-surface: #ffffff;
  --color-text: #1a1a1a;
  --color-text-muted: #595959;
  --color-border: #d9dee3;
  --color-accent: #1f4e79;
  --color-alert: #c0392b;
  --font-heading: "Poppins", sans-serif;
  --font-body: "Inter", system-ui, -apple-system, sans-serif;
  --radius-lg: 20px;
  --radius-md: 14px;
  --radius-sm: 8px;
  --content-width: clamp(320px, 92vw, 1100px);
}

[data-theme="dark"] {
  --color-bg: #0f1420;
  --color-surface: #171e2c;
  --color-text: #f5f5fb;
  --color-text-muted: #a9b0c3;
  --color-border: #2a3245;
  --color-accent: #6fa8dc;
}

```

8.1 Responsive Breakpoints

Breakpoints are implemented as plain CSS media queries within each component's module file (no CSS-in-JS), verified at the five sizes required by the evaluation criteria:

- 320px — small mobile
- 480px — large mobile
- 768px — tablet
- 1024px — small desktop / landscape tablet
- 1200px — desktop

9. Feature-to-Architecture Mapping

Maps each graded requirement to where it is implemented, so coverage can be checked at a glance during development and review.

Requirement	Implemented In
Real-time current conditions	components/CurrentConditions + hooks/useWeather + services/weatherApi
Hourly / daily forecast toggle	components/ForecastView (+ ForecastToggle, ForecastCard)
Geolocation detection	hooks/useGeolocation
Location search	components/SearchBar + SearchResultsList + services/weatherApi (search endpoint)
Severe weather alerts	components/AlertBanner + utils/weatherAlerts + Web Notifications API

Multiple saved locations	components/LocationTabs + hooks/useSavedLocations
Theme customization	hooks/useTheme + design tokens in src/index.css + pages/Settings
Unit customization	hooks/useUnits + utils/formatWeather + pages/Settings
Offline access	hooks/useWeather cache fallback + weather-cache localStorage key
Background-process notifications	components/Toast
Responsive layout	Per-component .module.css media queries + --content-width token

10. Tooling Configuration

10.1 package.json — scripts & core dependencies

```
{
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.x",
    "react-dom": "^19.x",
    "react-router-dom": "^6.x",
    "react-icons": "^5.x"
  },
  "devDependencies": {
    "@eslint/js": "^10.x",
    "@types/react": "^19.x",
    "@types/react-dom": "^19.x",
    "@vitejs/plugin-react": "^6.x",
    "eslint": "^10.x",
    "eslint-plugin-react-hooks": "^7.x",
    "eslint-plugin-react-refresh": "^0.5.x",
    "globals": "^17.x",
    "typescript": "~6.0.x",
    "typescript-eslint": "^8.x",
    "vite": "^8.x"
  }
}
```

10.2 Other Config Files

- vite.config.ts — standard Vite + @vitejs/plugin-react setup, no custom aliasing needed
- eslint.config.js — flat config: js recommended + typescript-eslint recommended + react-hooks + react-refresh (vite), same as both reference projects
- tsconfig.json / tsconfig.app.json / tsconfig.node.json — split config; app config targets ES2023/DOM, jsx: react-jsx, noUnusedLocals and noUnusedParameters enabled
- react-router-dom pinned to v6 per the task brief, for consistency across submissions

11. Appendix — Prompt Template for Scaffolding

For consistency with the two reference documents, this is the reusable prompt block for asking Claude (or another assistant) to scaffold the project from this specification.

Generate a new React + TypeScript + Vite project following the structure, tooling, and conventions in this documentation:

- Vite + React + TypeScript, CSS Modules per component, no CSS-in-JS.
- One folder per component under `src/components/<ComponentName>/`, PascalCase, each with `<ComponentName>.tsx` + `<ComponentName>.module.css`.
- `pages/` folder for Home (weather dashboard) and Settings (theme/units).
- `App.tsx` is a thin shell composing the router + top-level layout only.
- Design tokens (colors, fonts, radii, content width) defined once as CSS custom properties in `src/index.css`, including a `[data-theme="dark"]` override, consumed via `var(--token)` everywhere.
- Reusable primitives (Button, ToggleSwitch, Text) live in their own component folders and are imported by sections, never duplicated.
- All persisted state (saved locations, active location, unit system, theme, weather cache) goes through one generic `useLocalStorage<T>` hook.
- Data fetching goes through `services/weatherApi.ts`, consumed by `hooks/useWeather`.
- `react-router-dom v6` for routing.
- Same `eslint.config.js` flat config and split `tsconfig` as the reference projects.

Reference Documents Used

- Task brief PDF — objective, scenario, requirements, and evaluation criteria
- Links Vault Structure Guide — hooks/utils/types separation, one-folder-per-component + CSS Modules, no third-party CSS
- ui-challenge-one Project Structure Template — section-based composition, design-token system, reusable primitives, tooling configuration

12. Appendix B — Weather API Service Implementation (Working Code)

This appendix contains the actual, working implementation of the WeatherAPI.com service layer referenced throughout this document (`services/weatherApi.ts`) and its supporting types (`types/weather.ts`, `types/location.ts`). Unlike the illustrative snippets in earlier sections, this code is complete and type-checks cleanly - it is not a placeholder.

To use it: create a free API key at weatherapi.com/signup.aspx and add it to a `.env` file at the project root as `VITE_WEATHERAPI_KEY=your_key_here`. This has not been exercised against a live key/response, so a real test call is recommended before building UI on top of it.

12.1 `src/types/weather.ts`

```
// src/types/weather.ts

export interface WeatherCondition {
  text: string;
  icon: string;
  code: number;
```

```

}

export interface CurrentWeather {
  lastUpdatedEpoch: number;
  tempC: number;
  tempF: number;
  isDay: boolean;
  condition: WeatherCondition;
  windKph: number;
  windMph: number;
  windDir: string;
  pressureMb: number;
  precipMm: number;
  humidity: number;
  cloud: number;
  feelsLikeC: number;
  feelsLikeF: number;
  uv: number;
  gustKph: number;
}

export interface HourlyForecast {
  id: string;
  timeEpoch: number;
  time: string;
  tempC: number;
  tempF: number;
  isDay: boolean;
  condition: WeatherCondition;
  chanceOfRain: number;
  chanceOfSnow: number;
  willItRain: boolean;
  willItSnow: boolean;
  windKph: number;
  humidity: number;
}

export interface ForecastDay {
  id: string;
  date: string;
  dateEpoch: number;
  maxTempC: number;
  minTempC: number;
  maxTempF: number;
  minTempF: number;
  avgHumidity: number;
  maxWindKph: number;
  totalPrecipMm: number;
  condition: WeatherCondition;
  uv: number;
  sunrise: string;
  sunset: string;
  hours: HourlyForecast[];
}

export type AlertSeverity = "minor" | "moderate" | "severe" | "extreme";

export interface WeatherAlert {
  id: string;
  headline: string;
  severity: AlertSeverity;
  event: string;
}

```

```

    effective: string;
    expires: string;
    description: string;
  }

  export interface WeatherLocationInfo {
    name: string;
    region: string;
    country: string;
    lat: number;
    lon: number;
    tzId: string;
    localtime: string;
  }

  export interface WeatherSnapshot {
    location: WeatherLocationInfo;
    current: CurrentWeather;
    forecastDays: ForecastDay[];
    alerts: WeatherAlert[];
    fetchedAtEpoch: number;
  }

```

12.2 src/types/location.ts

```

// src/types/location.ts

export interface SearchResult {
  id: number;
  name: string;
  region: string;
  country: string;
  lat: number;
  lon: number;
  /** Value to pass back into the API as the `q` param, e.g. "lat,lon" */
  queryValue: string;
}

export interface SavedLocation {
  id: string;
  label: string;
  queryValue: string;
  isCurrentLocation: boolean;
}

```

12.3 src/services/weatherApi.ts

Provides `fetchCurrentAndForecast()` (current conditions + forecast + alerts in one call), `searchLocations()` (autocomplete for the search bar), and `coordsToQueryValue()` (formats geolocation coordinates for the API's `q` parameter), plus full mapping from WeatherAPI's raw `snake_case` JSON into this project's `camelCase` types.

```

// src/services/weatherApi.ts
//

```

```
// Thin, typed fetch wrapper around the WeatherAPI.com REST API.
// Docs: https://www.weatherapi.com/docs/
//
// Requires an API key from https://www.weatherapi.com/signup.aspx
// Set it in a `.env` file at the project root (Vite reads VITE_-prefixed vars):
//   VITE_WEATHERAPI_KEY=your_key_here

import type {
  WeatherSnapshot,
  ForecastDay,
  HourlyForecast,
  WeatherAlert,
  AlertSeverity,
  CurrentWeather,
  WeatherLocationInfo,
} from "../types/weather";
import type { SearchResult } from "../types/location";

const BASE_URL = "https://api.weatherapi.com/v1";
const API_KEY = import.meta.env.VITE_WEATHERAPI_KEY as string | undefined;

export class WeatherApiError extends Error {
  constructor(message: string, public status?: number) {
    super(message);
    this.name = "WeatherApiError";
  }
}

// Reads the API key from the environment and throws a clear, actionable
// error immediately if it's missing, instead of letting a later fetch
// fail with a vague 401.
function assertApiKey(): string {
  if (!API_KEY) {
    throw new WeatherApiError(
      "Missing VITE_WEATHERAPI_KEY. Add it to your .env file before calling the weather API."
    );
  }
  return API_KEY;
}

// Builds the full request URL (base + path + key + query params), performs
// the fetch, and normalizes both network failures and non-OK HTTP responses
// into a single WeatherApiError so callers only need one catch path.
async function requestJson<T>(path: string, params: Record<string, string>): Promise<T> {
  const key = assertApiKey();
  const url = new URL(`${BASE_URL}${path}`);
  url.searchParams.set("key", key);
  Object.entries(params).forEach(([name, value]) => url.searchParams.set(name, value));

  let response: Response;
  try {
    response = await fetch(url.toString());
  } catch {
    throw new WeatherApiError("Network request failed. Check your connection and try again.");
  }

  if (!response.ok) {
    // WeatherAPI.com returns { error: { code, message } } on failure
    let message = `Weather API request failed with status ${response.status}`;
  }
}
```

```

    try {
      const body = await response.json();
      if (body?.error?.message) message = body.error.message;
    } catch {
      // response body wasn't JSON – keep the generic message
    }
    throw new WeatherApiError(message, response.status);
  }

  return (await response.json()) as T;
}

// -----
// Response shape (subset of the fields WeatherAPI.com actually returns)
// -----

interface RawCondition {
  text: string;
  icon: string;
  code: number;
}

interface RawLocation {
  name: string;
  region: string;
  country: string;
  lat: number;
  lon: number;
  tz_id: string;
  localtime: string;
}

interface RawCurrent {
  last_updated_epoch: number;
  temp_c: number;
  temp_f: number;
  is_day: number;
  condition: RawCondition;
  wind_kph: number;
  wind_mph: number;
  wind_dir: string;
  pressure_mb: number;
  precip_mm: number;
  humidity: number;
  cloud: number;
  feelslike_c: number;
  feelslike_f: number;
  uv: number;
  gust_kph: number;
}

interface RawHour {
  time_epoch: number;
  time: string;
  temp_c: number;
  temp_f: number;
  is_day: number;
  condition: RawCondition;
  chance_of_rain: number;
  chance_of_snow: number;
  will_it_rain: number;
  will_it_snow: number;
}

```

```

    wind_kph: number;
    humidity: number;
  }

  interface RawForecastDay {
    date: string;
    date_epoch: number;
    day: {
      maxtemp_c: number;
      mintemp_c: number;
      maxtemp_f: number;
      mintemp_f: number;
      avghumidity: number;
      maxwind_kph: number;
      totalprecip_mm: number;
      condition: RawCondition;
      uv: number;
    };
    astro: { sunrise: string; sunset: string };
    hour: RawHour[];
  }

  interface RawAlert {
    headline: string;
    severity: string;
    event: string;
    effective: string;
    expires: string;
    desc: string;
  }

  interface RawForecastResponse {
    location: RawLocation;
    current: RawCurrent;
    forecast: { forecastday: RawForecastDay[] };
    alerts?: { alert: RawAlert[] };
  }

  interface RawSearchResult {
    id: number;
    name: string;
    region: string;
    country: string;
    lat: number;
    lon: number;
  }

  // -----
  // Mapping: raw API JSON -> app-facing types (src/types/weather.ts)
  // -----

  // WeatherAPI.com's alert severity strings aren't a fixed enum, so this
  // downgrades whatever text it sends into our four-value AlertSeverity type,
  // defaulting to "minor" for anything unrecognized.
  function normalizeSeverity(raw: string): AlertSeverity {
    const value = raw.toLowerCase();
    if (value.includes("extreme")) return "extreme";
    if (value.includes("severe")) return "severe";
    if (value.includes("moderate")) return "moderate";
    return "minor";
  }
}

```



```

// Converts one raw hourly entry (snake_case, WeatherAPI's shape) into our
// camelCase HourlyForecast, including deriving a stable id from the epoch.
function mapHour(hour: RawHour): HourlyForecast {
  return {
    id: `${hour.time_epoch}`,
    timeEpoch: hour.time_epoch,
    time: hour.time,
    tempC: hour.temp_c,
    tempF: hour.temp_f,
    isDay: hour.is_day === 1,
    condition: hour.condition,
    chanceOfRain: hour.chance_of_rain,
    chanceOfSnow: hour.chance_of_snow,
    willItRain: hour.will_it_rain === 1,
    willItSnow: hour.will_it_snow === 1,
    windKph: hour.wind_kph,
    humidity: hour.humidity,
  };
}

// Converts one raw forecast day into our ForecastDay shape, flattening the
// nested day/astro objects and mapping all of its hourly entries via mapHour.
function mapForecastDay(day: RawForecastDay): ForecastDay {
  return {
    id: day.date,
    date: day.date,
    dateEpoch: day.date_epoch,
    maxTempC: day.day.maxtemp_c,
    minTempC: day.day.mintemp_c,
    maxTempF: day.day.maxtemp_f,
    minTempF: day.day.mintemp_f,
    avgHumidity: day.day.avghumidity,
    maxWindKph: day.day.maxwind_kph,
    totalPrecipMm: day.day.totalprecip_mm,
    condition: day.day.condition,
    uv: day.day.uv,
    sunrise: day.astro.sunrise,
    sunset: day.astro.sunset,
    hours: day.hour.map(mapHour),
  };
}

// Converts the raw "current conditions" object into our camelCase
// CurrentWeather type.
function mapCurrent(current: RawCurrent): CurrentWeather {
  return {
    lastUpdatedEpoch: current.last_updated_epoch,
    tempC: current.temp_c,
    tempF: current.temp_f,
    isDay: current.is_day === 1,
    condition: current.condition,
    windKph: current.wind_kph,
    windMph: current.wind_mph,
    windDir: current.wind_dir,
    pressureMb: current.pressure_mb,
    precipMm: current.precip_mm,
    humidity: current.humidity,
    cloud: current.cloud,
    feelsLikeC: current.feelslike_c,
    feelsLikeF: current.feelslike_f,
    uv: current.uv,
  };
}

```

```

    gustKph: current.gust_kph,
  };
}

// Converts the raw location block into our WeatherLocationInfo type
// (mainly renaming tz_id to tzId for naming-convention consistency).
function mapLocation(location: RawLocation): WeatherLocationInfo {
  return {
    name: location.name,
    region: location.region,
    country: location.country,
    lat: location.lat,
    lon: location.lon,
    tzId: location.tz_id,
    localtime: location.localtime,
  };
}

// Converts one raw alert into our WeatherAlert type. WeatherAPI.com doesn't
// give alerts a stable id, so one is synthesized from the event name + its
// index in the list.
function mapAlert(alert: RawAlert, index: number): WeatherAlert {
  return {
    id: `${alert.event}-${index}`,
    headline: alert.headline,
    severity: normalizeSeverity(alert.severity),
    event: alert.event,
    effective: alert.effective,
    expires: alert.expires,
    description: alert.desc,
  };
}

// Assembles the full WeatherSnapshot from a raw /forecast.json response by
// combining all the individual mappers above, and stamps it with the time
// it was fetched (used later for cache-freshness checks).
function mapSnapshot(raw: RawForecastResponse): WeatherSnapshot {
  return {
    location: mapLocation(raw.location),
    current: mapCurrent(raw.current),
    forecastDays: raw.forecast.forecastday.map(mapForecastDay),
    alerts: (raw.alerts?.alert ?? []).map(mapAlert),
    fetchedAtEpoch: Math.floor(Date.now() / 1000),
  };
}

// -----
// Public API
// -----

/**
 * Fetches current conditions + forecast + alerts for a location in one call.
 *
 * @param locationQuery - City name ("London"), "lat,lon" pair, US zip, or UK postcode.
 *   See https://www.weatherapi.com/docs/#intro-request for all accepted formats.
 * @param days - Number of forecast days to request (1-14, plan-dependent). Defaults to 7.
 */
export async function fetchCurrentAndForecast(
  locationQuery: string,
  days = 7
): Promise<WeatherSnapshot> {
  const raw = await requestJson<RawForecastResponse>("/forecast.json", {

```

```

    q: locationQuery,
    days: String(days),
    aqi: "no",
    alerts: "yes",
  });
  return mapSnapshot(raw);
}

/**
 * Autocomplete/search for locations matching a partial query string.
 * Used by the SearchBar component to populate SearchResultsList.
 */
export async function searchLocations(partialQuery: string): Promise<SearchResult[]> {
  if (!partialQuery.trim()) return [];

  const raw = await requestJson<RawSearchResult[]>("/search.json", { q: partialQuery });

  return raw.map((result) => ({
    id: result.id,
    name: result.name,
    region: result.region,
    country: result.country,
    lat: result.lat,
    lon: result.lon,
    queryValue: `${result.lat},${result.lon}`,
  })));
}

/**
 * Convenience helper for resolving a browser Geolocation position
 * into a query string the API accepts directly.
 */
export function coordsToQueryValue(latitude: number, longitude: number): string {
  return `${latitude},${longitude}`;
}

```